```
var projection = d3.geo.mercator()
    .scale(120)
    .translate([width / 2, height / 2]);

var center = projection([12, 42]);

var path = d3.geo.path()
    .projection(projection);

   var featureSize = d3.extent(countries.features, function(d) {
                      return path.area(d);
                     });
   var countryColor = d3.scale.quantize()
                      .domain(featureSize)
                      .range(colorbrewer.Reds[7]);

var zoom = d3.behavior.zoom()
    .scale(projection.scale() * 2 * Math.PI)
    .translate([width - center[0], height - center[1]])
    .on("zoom", redraw);

    d3.select("svg").call(zoom);
    redraw();

d3.select("svg").selectAll("path.countries").data(countries.features)
    .enter()
    .append("path")
    .attr("d", path)
    .attr("class", "countries")
    .style("fill", function(d) {return countryColor(path.area(d))})
    .style("stroke-width", 1)
    .style("stroke", "black")
    .style("opacity", .5)

    d3.select("svg").selectAll("circle").data(cities)
               .enter()
            .append("circle")
               .attr("class", "cities")
               .attr("r", 3)
               .attr("cx", function(d) {
                     return projection([d.x,d.y])[0];
               })
            .attr("cy", function(d) {
     return projection([d.x,d.y])[1];
});

    function redraw() {
      var tiles = tile
      .scale(zoom.scale())
      .translate(zoom.translate())
      ();

      var image = d3.select("#tiles")
      .attr("transform", "scale(" + tiles.scale + ")translate(" +
          tiles.translate + ")")
      .selectAll("image")
      .data(tiles, function(d) { return d; });
```